A3-PS013-TC067

Incident Timeline and Evidence Collection for Small Teams

Manasvi Manoj Yeole
Gayatri Sanjay Karanjkar

AIORI Hackathon 3

October 11, 2026

Hash

Normalise

Correct skew

Report

Abstract: The Problem



Scattered

Evidence sits on several servers and applications



Messy

Every source writes its own log format



Out of sync

Time zones differ, clocks drift, syslog has no year



Fragile

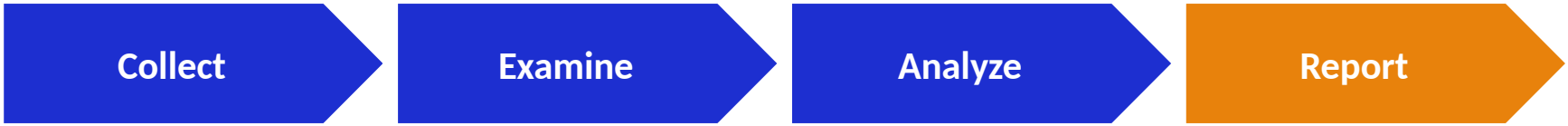
Careless copying or edits weaken the evidence

Our answer

A lightweight tool that hashes evidence on collection, converts every timestamp to UTC, corrects clock skew between hosts, and builds a searchable timeline with an exportable report.

Built for solo defenders and small IT teams.

Background



NIST SP 800-86 [1] stages (top) and the part of our tool that covers each one

Small teams lack a SIEM or forensic suite and fall back on grep and spreadsheets.

Integrity rests on hashing at collection (SHA-256 [5]) and keeping the originals untouched.

Time is the hardest part: zones differ, clocks drift and syslog lines carry no year.

Out of scope: live endpoint takeover, malware removal, forensic certification

Literature & RFC Review

Source	What it provides	How we use it
NIST SP 800-86 [1]	Forensics in incident response: collect, examine, analyze, report	Defines our pipeline stages
RFC 3164 / 5424 [2][3]	Syslog formats; 3164 timestamps carry no year or zone	Year and zone are declared inputs
RFC 3339 [4]	Internet timestamp format	Normalised UTC with milliseconds
FIPS 180-4 [5]	SHA-256 secure hash standard	Evidence hashes and manifest
Plaso, Timesketch [6][7]	Full timeline platforms	Heavy for small teams; we stay lightweight
Nginx log_format [8]	Access log with its own UTC offset	Offset on the line wins over declared zone

Proposed Solution

1 Collect and hash

SHA-256 on collection. Originals copied read-only; the manifest is hashed too.

2 Parse

auth.log and Nginx parsers. Every unparsed line is stored with a reason.

3 Normalise

All timestamps converted to UTC. The original text is kept beside it.

4 Correct skew

Events seen on two hosts give a median clock offset with a confidence note.

5 Search

Chronological timeline with filters and SQLite full-text search.

6 Report

HTML, JSON and CSV with hashes, skew, gaps and limitations.

Optimization Proposed

Aspect	Manual: grep + spreadsheet	This tool
Evidence integrity	No hash; copies edited freely	**SHA-256 manifest; verify detects tampering**
Original vs working copy	Mixed together	**Originals read-only; normalised copy separate**
Timestamps	Mixed zones; syslog has no year	**Every event in UTC; original text kept**
Clock skew	Estimated by hand	**Anchor median with a confidence note**
Unparsed lines	Silently missed	**Stored with a reason and reported**
Triage time per question	Measured manually (slide 15)	**Measured with the tool (slide 15)**

Solution Architecture & Design

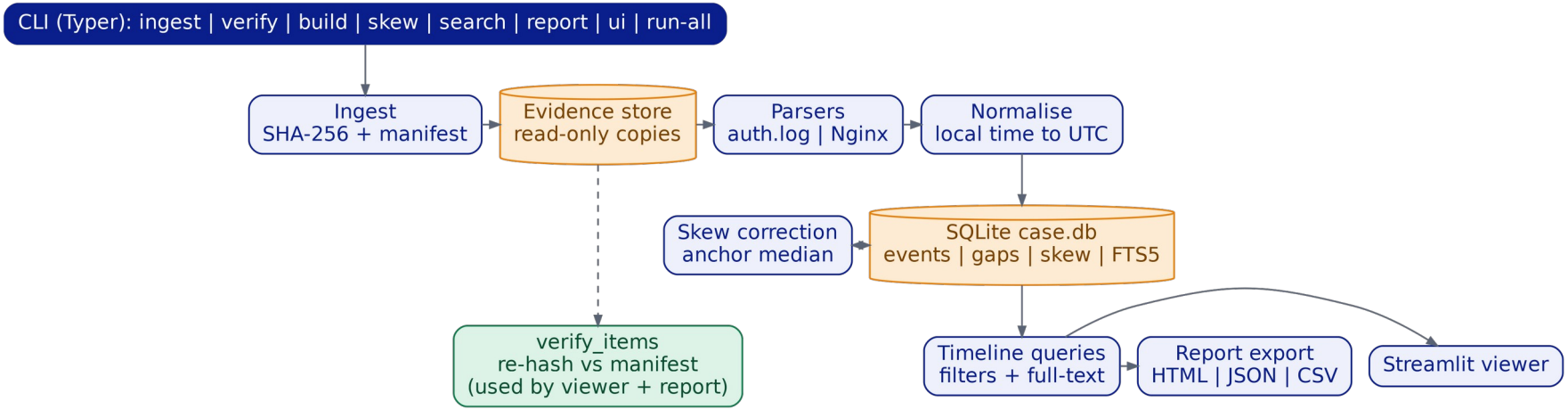


Figure: System architecture (component view)

The evidence store is write-once; later stages read copies only

One SQLite file per case, with full-text search and no server

verify_items is shared by the viewer and the report

Data Flow Diagram

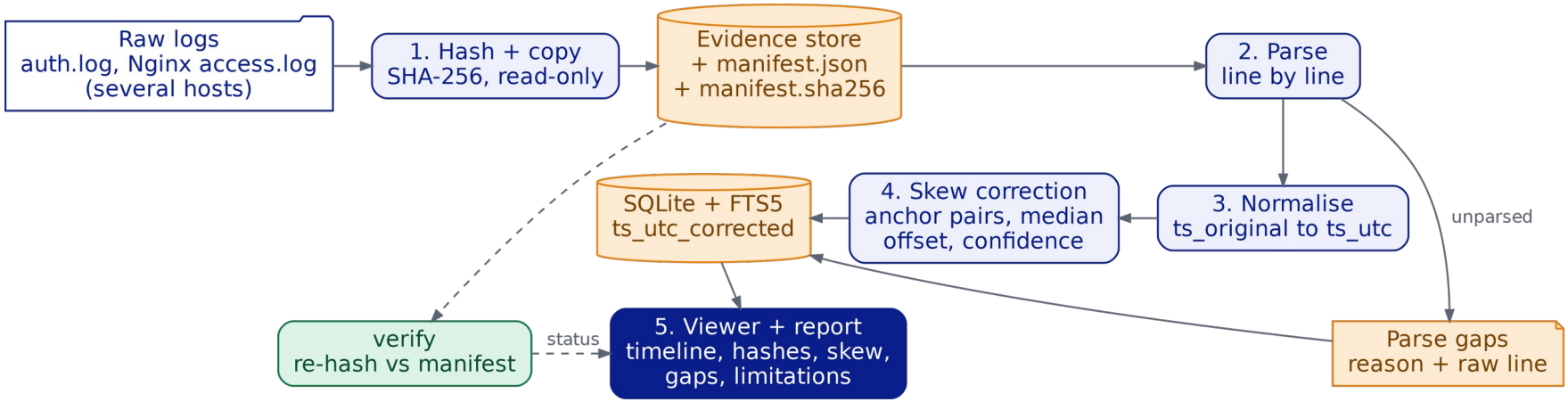


Figure: Data Flow Diagram (DFD)

Originals are never modified

Nothing is silently dropped: unparsed lines are stored with a reason

Corrected time is stored next to the original, not over it

Prototype (1/4): Evidence Workflow in the CLI

run-all, scenario 3 (condensed real output)

```
$ forensic run-all
... scenario s3_clock_skew
OK manifest.json
OK evidence/web01__auth.log
OK evidence/web01__access.log
OK evidence/db01__auth.log
OK evidence/db01__access.log
ID | Filename   | Host  | Parsed | Gaps | % Parsed
1  | auth.log   | web01 | 10     | 0    | 100.0%
2  | access.log | web01 | 12     | 0    | 100.0%
3  | auth.log   | db01  | 2      | 0    | 100.0%
4  | access.log | db01  | 11     | 0    | 100.0%
db01: anchor_median offset -421.5s (high, n=10)
s3_clock_skew: 35 events parsed, 0 gaps
```

One command: verify, hash, parse, normalise and skew-correct three synthetic incidents.

verify detects tampering (hashes shortened)

```
$ forensic verify --case cases/s3_clock_skew
OK manifest.json
OK evidence/web01__auth.log
Verification passed.

# append one byte to a stored log
$ forensic verify --case cases/s3_clock_skew
OK evidence/web01__access.log
TAMPERED evidence/db01__auth.log
  expected=2fbc3218...  actual=77df1485...
OK evidence/db01__access.log
exit code 1
```

search the timeline

```
$ forensic search --case cases/s2_web_attack "UNION"
2026-10-05T14:09:33.000Z | web01
web_sqli_attempt | 203.0.113.77
GET /products.php?id=1' UNION SELECT 1,2
```

Prototype (2/4): Timeline and Report

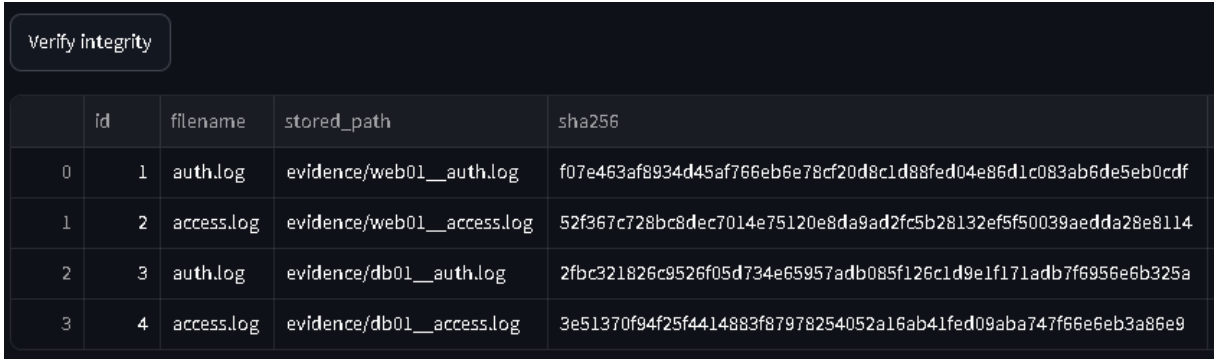
INSERT SCREENSHOT
viewer_timeline.png

Timeline tab: filters, events per minute (UTC), skew-corrected times

INSERT SCREENSHOT
report_overview.png

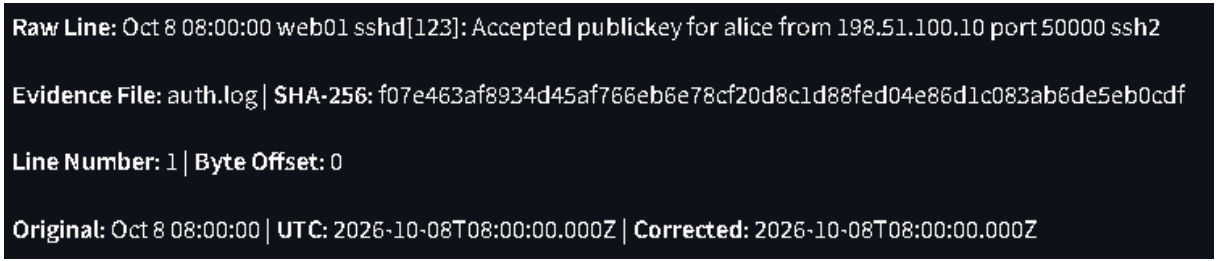
Incident report: summary, key events, hashes with verification status, skew, gaps, limitations

Prototype (3/4): Evidence and Event Trace



	id	filename	stored_path	sha256
0	1	auth.log	evidence/web01__auth.log	f07e463af8934d45af766eb6e78cf20d8c1d88fed04e86d1c083ab6de5eb0cdf
1	2	access.log	evidence/web01__access.log	52f367c728bc8dec7014e75120e8da9ad2fc5b28132ef5f50039aedda28e8114
2	3	auth.log	evidence/db01__auth.log	2fbc321826c9526f05d734e65957adb085f126c1d9e1f171adb7f6956e6b325a
3	4	access.log	evidence/db01__access.log	3e51370f94f25f4414883f87978254052a16ab41fed09aba747f66e6eb3a86e9

Evidence tab: every log file with its SHA-256



Event detail: raw line, file, hash, line and byte offset, original vs corrected time

Hash recorded at collection

Verify integrity re-hashes every stored file

Every event links to its file and line

Prototype (4/4): Parsing Gaps and Clock Skew

filename	host	total_lines	blank_lines	parsed	gaps	parsed_pct
auth.log	web01	10	0	10	0	100
access.log	web01	12	0	12	0	100
auth.log	db01	2	0	2	0	100
access.log	db01	11	0	11	0	100

Parsing gaps tab: per-file coverage. Scenario 3 parses 100% of its 35 lines

	host	reference_host	offset_s	method	anchor_count	stdev_s
0	web01	web01	0	none	0	None
1	db01	web01	-421.5	anchor_median	10	0.8718

Clock skew tab: db01 offset -421.5 s, method anchor_median, 10 anchors, stdev 0.87 s (confidence note in the last column: high)

Nothing is silently dropped

Each correction shows method, anchors and confidence

The reference host keeps offset 0

Results: What We Measured

18 / 18
ground-truth steps matched within 1.5 s

-421.5 s
estimated skew; true value -420 s (10 anchors)

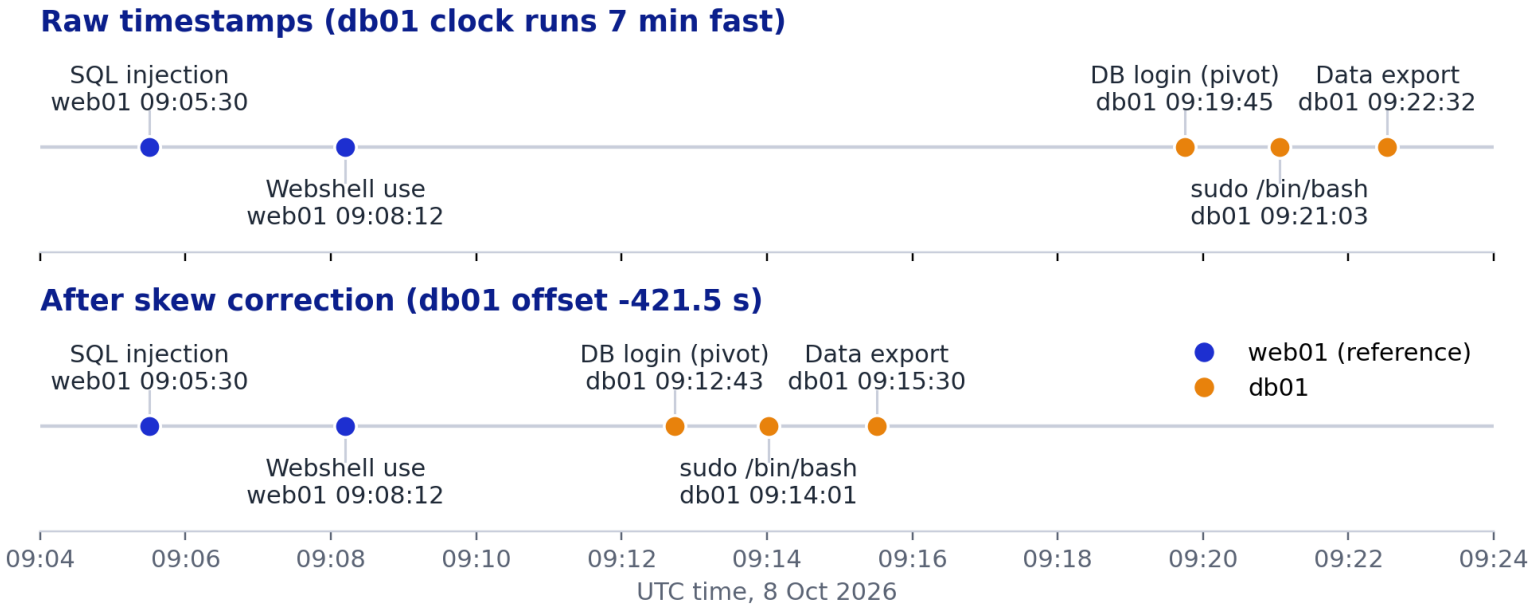
4 | 5 | 0
parsing gaps recorded in S1 | S2 | S3

26
automated tests passing

Scenario	Log lines	Parsed events	Gaps	Parsed %
S1 SSH brute force	169	165	4	97.6%
S2 Web attack	111	106	5	96.4%
S3 Clock skew	35	35	0	100.0%

S2 gaps include 1 blank line; parsed % is over non-blank lines. Data: three synthetic incidents.

Clock Skew: Scenario 3



Pivot after webshell: **raw 11 min 33 s** | corrected **4 min 31 s** | ground truth 4 min 33 s

Triage Time vs Manual Baseline

S2 and overall rows pending: S2 is being re-timed with corrected questions

Scenario	Manual median (s)	Tool median (s)	Accuracy manual	Accuracy tool
S1 SSH brute force	34.8	9.5	100%	100%
S2 Web attack	*pending*	*pending*	*pending*	*pending*
S3 Clock skew	78.2	11.2	66.7%	100%
All scenarios	*pending*	*pending*	*pending*	*pending*

Method: 6 timed sessions, 3 questions each. Manual = grep, sort and less only. Tool = CLI or viewer. Times recorded with evaluation/triage_timer.py. Small sample and testers who know the tool: results are indicative only.

Limitations & Scope

Limitations

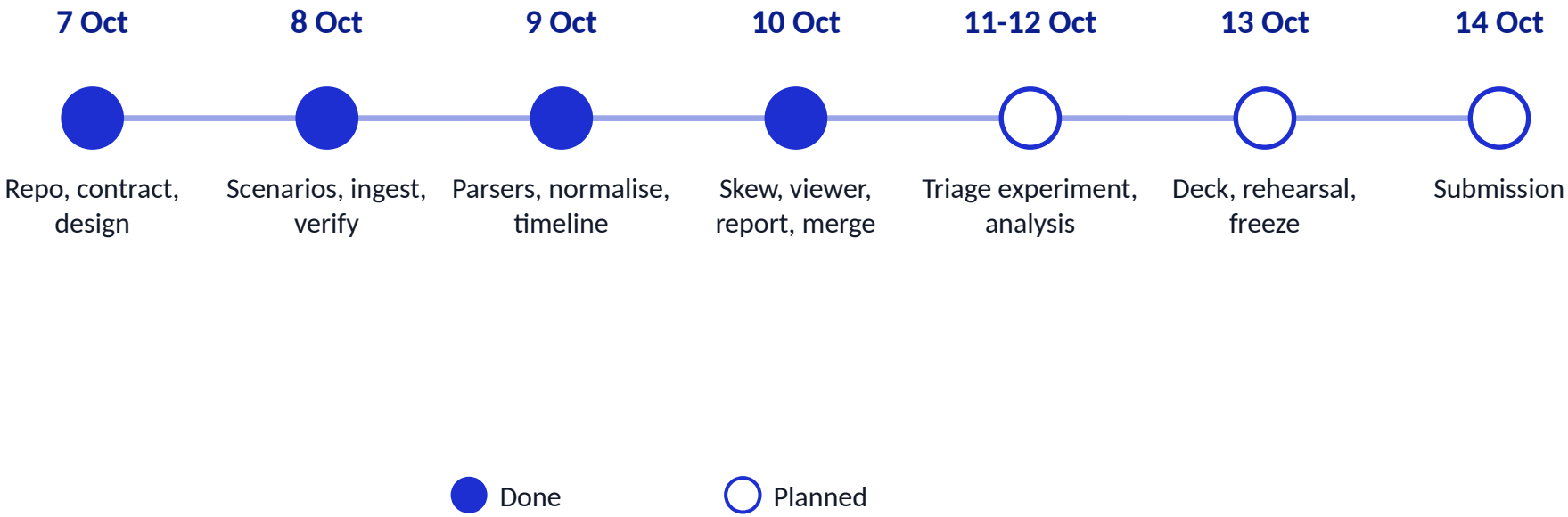
- Syslog year and time zone are declared by the operator
- Skew assumes paired events happen within about 5 s
- Daylight-saving ambiguity is not resolved
- Two log formats: auth.log and Nginx
- Evaluated on three synthetic incidents

Out of scope

- Live endpoint takeover
- Malware removal
- Forensic certification

The tool supports an investigation; it does not replace a certified forensic process.

Deployment Timeline



Bibliography

[1] K. Kent, S. Chevalier, T. Grance, H. Dang. Guide to Integrating Forensic Techniques into Incident Response. NIST SP 800-86, 2006.

[2] C. Lonvick. The BSD Syslog Protocol. RFC 3164, 2001.

[3] R. Gerhards. The Syslog Protocol. RFC 5424, 2009.

[4] G. Klyne, C. Newman. Date and Time on the Internet: Timestamps. RFC 3339, 2002.

[5] NIST. Secure Hash Standard (SHS). FIPS 180-4, 2015.

[6] Plaso (log2timeline) documentation. github.com/log2timeline/plaso

[7] Timesketch documentation. github.com/google/timesketch

[8] Nginx documentation. ngx_http_log_module (log_format).



Thank You for Your Attention!

Questions?